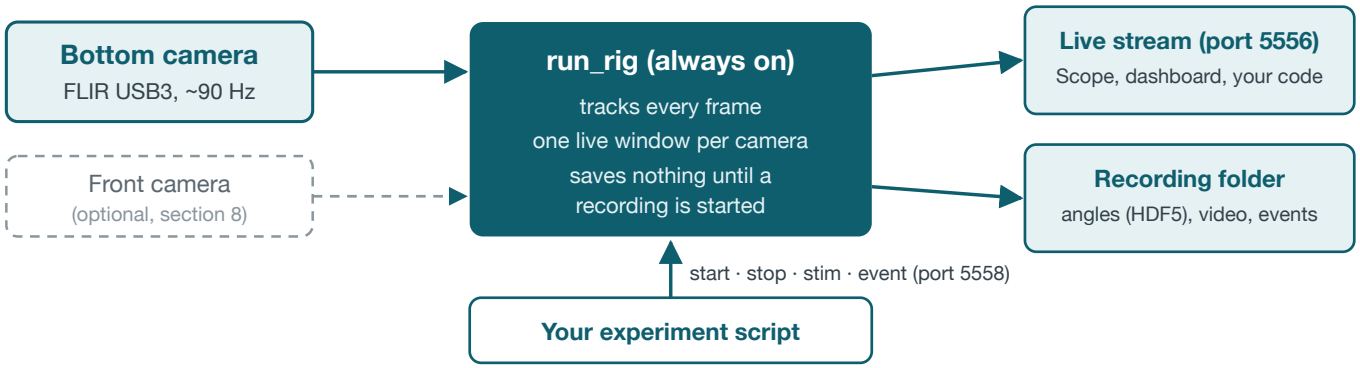
Kinefly2: Getting Started

Install, set up and record with Kinefly2. This guide is written for a rig with only the bottom camera.

For new users. Code version 9ccc802, October 2026. The full reference is `README.md` in the repository; section numbers like (README §2.2) point there.

1 What Kinefly2 does

Kinefly2 tracks a tethered fly's wings, head and abdomen in camera images, in real time, at about 90 frames per second. It is a Python 3 version of the Dickinson-lab *Kinefly* and does not need ROS. Leave it running all day. Your experiment script tells it when to save a recording and which stimulus is on.



Cameras feed `run_rig`, which tracks continuously and publishes a live stream. A recording, with files on disk, exists only between a script's *start* and *stop*.

Measurement	Bottom camera only	With front camera	Column in the recording
Left / right wing stroke amplitude	**yes**	yes	`bottom/left_angle`, `right_angle`
L–R (steering), L+R	**yes**	yes	computed from the two wings
Head yaw	**yes (exact)**	yes	`bottom/head_yaw`
Abdomen bending	**yes**	yes	`bottom/abdomen_angle`
Head roll	**no**	yes (exact)	`front/head_roll`
Head pitch	**no**	yes, after a per-fly calibration	`front/head_pitch`
Hind legs	experimental	experimental	`bottom/leg_l3_angle`, `leg_r3_angle`

A rig with only the bottom camera is a complete setup. Everything in sections 2–7 works with one camera. Section 8 lists what changes if you add a front camera later.

2 What you need

Item	Requirement	Notes
Rig computer	Linux (Ubuntu) PC with USB3 ports, and a local disk with room for video (about 2–4 GB per camera per hour).	The FLIR camera driver runs here. A Mac can tune configs and analyze recorded videos, but cannot run live cameras.
Bottom camera	A monochrome FLIR (Teledyne) USB3 machine-vision camera supported by the Spinnaker SDK, ideally able to reach **90 fps** at your cropped image size. A slower camera also works (section 4).	Kinefly2 identifies the camera by its serial number, so the USB port it is plugged into doesn't matter.
Lens	Frames the whole fly, including the full wing stroke, inside the crop (e.g. 512 × 512 px).	A smaller crop allows a higher frame rate.
Illumination	An IR backlight **on the opposite side of the fly from the camera**, so the camera sees a dark fly on a bright field.	The bottom-view trackers expect this silhouette (`invertcolor: true`).
Fly holder	Your usual tether, held still relative to the camera.	Backgrounds and region positions assume the holder doesn't move.
Optional	A second FLIR camera in front of the fly, plus a GPIO cable between the two cameras.	For head roll and pitch (section 8).

3 Install (README §1)

On any computer (rig, or a Mac for analysis)

```
$ git clone https://github.com/hrli-neuro/Kinefly2.git
$ cd Kinefly2
$ conda create -n kinefly2 python=3.10        # 3.10: FLIR's PySpin wheels support only some versions
$ conda activate kinefly2
$ pip install numpy opencv-python pyyaml pyzmq h5py
$ pip install PySide6 pyqtgraph                # Kinefly2 Scope (trace viewer)
$ python tests/test_tracking.py                # expects wings r=1.000, head r~0.99; no camera needed
$ python tests/test_dual_pipeline.py
```

On the rig only: the FLIR driver

1. From flir.com → Spinnaker SDK, download the **Spinnaker SDK** .deb bundle for your Ubuntu version and the **spinnaker_python** (PySpin) wheel for **Python 3.10**.
2. Install the SDK following FLIR's README, including the udev rules. Then:

   ```
   $ conda activate kinefly2
   $ pip install spinnaker_python-*.whl
   $ python -c "import PySpin; print('PySpin OK')"
   ```

3. USB3 cameras need a larger kernel USB buffer. Run this **after every reboot**, or make it permanent through the kernel command line as FLIR's install notes describe:

   ```
   $ sudo sh -c 'echo 1000 > /sys/module/usbcore/parameters/usbfs_memory_mb'
   ```

4 Connect the bottom camera (README §2.1.1)

1. **Find the camera.** The command below prints the camera's serial number, model, link speed, resolution and maximum fps at the current crop, followed by a ready-to-paste `camera:` block.

```
$ python -m kinefly2.apps.list_cameras
```

2. **Paste the block** into `config/camera_bottom.yaml`, in place of the four `kind: video` lines at the top. With one camera, leave out the `trigger:` line.

```
camera:
  kind: pyspin
  serial: "22334455"              # from list_cameras: binds THIS camera to 'bottom'
  fps: 90
  exposure_us: 2000               # set manually; must be shorter than 1/fps (11111 us at 90 Hz)
  gain_db: 0
  roi: {width: 512, height: 512}  # crop to reach 90 fps (centred by default)
```

3. **Tell the rig there is only one camera.** In `config/rig.yaml`, switch the front camera off. Alternatively, add `--cameras bottom` to every `run_rig` command.

```
cameras:
  bottom: camera_bottom.yaml
  front:
    config: camera_front.yaml
    enabled: false                # bottom-only rig
primary: bottom
```

4. **Check the timing.** A healthy result is an interval median of ≈ 1000/fps ms (11.1 ms at 90 Hz), 0 dropped frames, and a track median of a few milliseconds.

```
$ python -m kinefly2.apps.check_latency config/camera_bottom.yaml --seconds 30
```

- **Exposure must be manual.** Auto-exposure changes brightness from frame to frame, and that moves the tracked edges.
- **If the frame rate is too low, shrink the `roi`.** A smaller crop is what buys frame rate.

If the camera can't reach 90 Hz, you don't need to change anything.

- Kinefly2 caps the request at the camera's maximum and uses the rate the camera reports.
- The startup banner shows that rate, e.g. `rate: bottom 72.3 fps (asked 90)`, and it is saved with the videos and recordings.
- Tracking works at any frame rate.

First day on real hardware. The live-camera code (crop, binning, trigger) has not yet run on a real FLIR camera (README §6). Before you trust an experiment to it:

- check the *max fps* that `list_cameras` reports;
- run `check_latency`;
- make a short test recording and look at it in the Scope.

5 Set up the tracking for a fly (README §2.5, §2.6, §2.8)

```
$ python -m kinefly2.apps.run_rig config/rig.yaml --scope
```

This opens a live window for the bottom camera, with a toolbar and draggable handles, and a Kinefly2 Scope window with the traces. **Each time you release the mouse, the regions are saved** to `config/camera_bottom.yaml`, so the next start begins from the current tuning.

1. **Background, with no fly in view.** Click `saveBG`. Then turn on subtraction for the wings (`subtract: LR`) and the abdomen (`subtract: A`).
   - The wings need it because faint wing blur only shows up against a subtracted background.
   - The abdomen needs it because otherwise the tracker reads the tether. In one test, abdomen noise dropped from 11° to 1.5° with subtraction on.
2. **Mount the fly, then click `autoplace`.** Every region moves to match the reference frame from the last tuning, so a fly mounted slightly differently needs at most a nudge. `reset` undoes it.
3. **Wings.** Drag each wing hinge onto its wing base.
   - The two line handles set the angular window, and the two circle handles set the inner and outer radius.
   - **0° is the line joining the two wing hinges**, so placing the hinges accurately sets the baseline.
   - With `symmetric` checked, editing one wing mirrors the other.
4. **Head: 4–6 landmarks.** Click `+H`, then click a feature; `−H` removes one. Where to put them:
   - **only in front of the neck hinge.** Points behind it land on the thorax and forelegs, which move on their own;
   - on outline corners such as antennae, proboscis or the neck notch, *not* inside a uniform dark patch;
   - spread apart, in an irregular, non-symmetric pattern.
5. **Abdomen.** Put its hinge at the abdomen base, and set the window so it covers the abdomen. The tracker uses the abdomen's two side edges.
6. **Zero.** When the fly is settled and holding a typical posture, click `zero`. The head angles are measured relative to this posture.
7. **Check.** The overlay must show no warnings (table below), and the Scope traces should move with the fly.

Toolbar

Button	What it does
`exit`	Quit (an active recording is closed cleanly)
`saveBG`	Current image becomes the background (no fly!)
`autoplace` / `reset`	Fit the regions to a new fly / undo
`zero`	Current head posture = 0
`record`	Start/stop a recording (template fields)
`+H` / `−H`	Add / remove a head landmark
`track: H A L R … X`	Parts on/off (X = aux intensity region)
`subtract: H A LR X`	Background subtraction per part
`invertcolor`	On for the bottom view (dark fly)
`symmetric`	Mirror left/right edits
`angles`	Extra readouts while tuning

Overlay warnings

Message	Fix
`2 edges (want 1)`	The sector catches a second object; narrow its angle window or radii
`NON−RIGID x px`	The landmarks are not moving together; move any on glints or behind the neck
`lmN BAD` (amber)	Move that landmark
red ring	The landmark is on a featureless spot; move it onto an edge or corner
`AMBIGUOUS LAYOUT`	The pattern is too symmetric; make it irregular
`only N landmarks`	Use at least 4
`search 10−>3`	The landmarks are crowded; spread them out

Hind legs are experimental. `leg_l3` and `leg_r3` are switched on in the current config, but they have not been shown to follow the leg. In tests they read the body margin instead (README §2.6). Turn them off with their `track` boxes, or check that `bottom/leg_*_n_edges` is 1 on almost every frame before you use them.

6 Record during an experiment (README §2.2–2.4)

The experiment template

Copy `experiments/default.yaml` to `experiments/<your project>.yaml` and fill it in:

- `data_root`, `genotype`, `fly_id`, `protocol`, `notes`;
- `record_video` (raw video lets you re-analyze later) and `events_port`.

Start the rig with it:

```
$ python -m kinefly2.apps.run_rig config/rig.yaml --exp experiments/myproject.yaml --scope
```

Each recording gets its own folder, named `<date>_<genotype>_<fly_id>_<protocol>`. The name gets a `_2`, `_3`, … suffix if it already exists, so nothing is overwritten.

Three ways to start and stop a recording

From	How
The camera window	Click `record`. It uses the template's fields, and a red **REC name m:ss** appears in the window.
A terminal	`python -m kinefly2.apps.ctl start --fly 017 --protocol YAMV` · `... ctl stop` · `... ctl status`
Your experiment script	The Python client, below. This is the recommended way, because the script can label every stimulus.

```
# needs only pyzmq; kinefly2/client.py can be copied next to your script
import time
from kinefly2.client import Kinefly2Client

k = Kinefly2Client('localhost')                # or the rig's IP if the script runs on another PC
print(k.status()['cameras'])                   # check: fps as expected, down: False

PATTERNS = [(1, 3.0), (2, 3.0), (3, 3.0)]      # (pattern ID, seconds)

with k.recording(fly_id='017', protocol='YAMV', notes='stripe fixation') as rec:
    k.stim(0)                                  # baseline: no pattern
    for trial, (pid, dur) in enumerate(PATTERNS):
        arena.show_pattern(pid)                # PLACEHOLDER: your arena's Python API call
        k.stim(pid)                            # frames from now on are labelled with pid
        k.event({'trial': trial, 'event': 'stim_on', 'pattern': pid})
        time.sleep(dur)
        arena.stop()                           # PLACEHOLDER
        k.stim(0)
        k.event({'trial': trial, 'event': 'stim_off'})
# leaving the block stops the recording, even if the script crashes
```

- `k.stim(value)` labels frames with the stimulus that is on.
  - Any number works, such as the pattern ID. It is **held until you change it**, so every row of `/merged/stim` says which stimulus was on at that frame.
  - Call it **right after** the arena call. The label is accurate to about ±1 frame (11 ms at 90 Hz).
  - The arena may take a moment to show a pattern after its call returns. If timing matters, measure that delay once.
- `k.event(dict)` logs anything you like, timestamped on the same clock as the frames. Events are kept only while a recording is running.
- **The stimulus label carries over between recordings**, so set `k.stim(...)` at the start of each one.
- **Other calls:**
  - `k.zero()` is the same as the `zero` button;

- `k.snapshot()` saves a PNG of the current frame;
- `k.status()` reports camera health and the recording in progress;
- `k.start(...)` / `k.stop()` work without a `with` block, and `stop()` returns a summary.

- **MATLAB** can send the same commands as JSON over ZMQ, e.g. `{"cmd":"stim","value":2}` to port 5558 (README §2.2).

Watching live

Use `--scope` on `run_rig`, as above, or `python -m kinefly2.apps.scope --live`. From another computer, use `python -m kinefly2.apps.scope --live tcp://<rig-ip>:5556`.

Without installing anything on the viewing computer, use the browser dashboard: run `python -m kinefly2.apps.dashboard` on the rig, then open `http://<rig>:8000`.

Stimulus changes and recording start/stop appear as markers on the traces.

7 Your data (README §2.3, §3.4)

```
~/Data/Kinefly2/260819/260819_XHCS_XHCS_017_YAMV/
  260819_XHCS_XHCS_017_YAMV_flystate.h5        angles, timestamps, stimulus code, events
  260819_XHCS_XHCS_017_YAMV_bottom.avi         raw video (record_video: true)
  260819_XHCS_XHCS_017_YAMV_bottom_frames.csv  frame id and timestamps of each video frame
  260819_XHCS_XHCS_017_YAMV_events.jsonl       your events, as received
  260819_XHCS_XHCS_017_YAMV_experiment.yaml    the settings used
  configs/                                     copies of the rig and camera configs used
```

HDF5 dataset (one row per frame)	Unit	Meaning
`/bottom/t_camera`	s	Time of each frame (camera clock); use it as the time axis
`/bottom/left_angle`, `right_angle`	rad	Wing stroke amplitude
`/bottom/head_yaw`	rad	Head yaw (exact from the ventral view)
`/bottom/abdomen_angle`	rad	Abdomen bending
`/bottom/seq`	–	Camera frame number; a gap means a dropped frame
`/merged/stim`	–	The `k.stim` value in force on that frame
`/events/t_host`, `/events/text`	s, text	Your events. `t_host` is on the same clock as `/bottom/t_host`

Angles are stored in **radians**; the GUI and Scope display degrees. **NaN** means the part was not found on that frame.

```
import h5py, numpy as np
f = h5py.File('…_flystate.h5', 'r')
t    = f['bottom/t_camera'][:]
L    = np.degrees(f['bottom/left_angle'][:])
R    = np.degrees(f['bottom/right_angle'][:])
yaw  = np.degrees(f['bottom/head_yaw'][:])
stim = f['merged/stim'][:]
events = f['events/text'].asstr()[:]
```

```
% MATLAB
L = h5read(file, '/bottom/left_angle');
stim = h5read(file, '/merged/stim');
```

View a file:
`python -m kinefly2.apps.scope <file>_flystate.h5`.
Use `C` for measurement cursors.

Re-track saved videos with other settings:
`python -m kinefly2.apps.analyze_videos`. It writes to a new folder and never changes the rig config (README §2.1.2).

8 Adding the front camera later (README §2.1.1, §2.3, §2.5)

- **Connect it.** Run `list_cameras`, paste the front camera's block into `config/camera_front.yaml`, and set `enabled: true` in `rig.yaml`.
- **Sync the two cameras.**
  - Use the same `fps` on both, with `trigger: {mode: primary}` on the bottom camera and `trigger: {mode: secondary}` on the front.
  - Wire the primary's output line to the secondary's trigger input, following FLIR's GPIO pinout for your model.
  - Without the trigger, the front frames can be up to one frame off.
- **Front head landmarks** go on the eye edges, the bright patch between the eyes, and the antenna bases. Drag the `+pitch` arrow along the fly's dorsoventral axis.
- **Pitch and yaw from the front camera are in pixels until you calibrate each fly.** After the fly's first recording, run the command below. Then either restart `run_rig`, which reads the template only at start-up, or pass the value directly with `k.start(..., lever_px={'front': 48.2})`. That fly's later recordings then report degrees.

  ```
  $ python -m kinefly2.apps.calibrate_fly <recording_folder> --write experiments/myproject.yaml
  ```

- **STALE badge.** If the front camera falls behind by more than 55 ms, its values are saved as NaN instead of being repeated, and the windows show a **STALE** badge.